

Document number: P0675R0

Date: 2017-06-19

Reply-to: John McFarlane, fixed-point@john.mcfarlane.name

Audience: SG6, SG14, LEWG

Numeric Traits for Type Composition

Introduction

This paper identifies a core set of trait-like definitions necessary to support arbitrary numeric types.

It adds to numeric traits proposal, [P0437], and aims to facilitate the compositional style of numeric type generation described in [P0554]. It contains replacements for the two definitions introduced in [P0381]. An example of a type which greatly benefits from this support is the fixed-point type detailed in [P0037].

Motivation

The aim is to achieve a high degree of interoperability between fundamental numeric types (such as `int`), existing custom numeric types (such as `chrono::duration` and those found in [Boost.Multiprecision]) and future types (such as through the Numerics TS [P0101]) which will be able to apply the types described here.

Interoperability is often assumed to mean the ability to mix different types in algebraic expressions. However, it is also advantageous to compose types from one another (as described in [P0554]). For example, to compose custom types from fundamental integers:

```
// use an unsigned 16-bit integer to approximate a real number with 2 integer and 14 fractional digits
auto pi = fixed_point<uint16_t, -14>{3.141};
assert(pi > 3.1 && pi < 3.2);

// use int to store value gained using accurate rounding mode
auto num_children = rounded_integer<int>{2.6};
assert(num_children == 3);
```

The versatility of such types is increased if they can be composed from one another:

```
// 8-bit type with good rounding characteristics and resolution of 1/16
auto num = fixed_point<rounded_integer<uint8_t>, -4>{15.9375};
```

In order to be generic, these custom types often require information about the underlying type with which they are instantiated. For instance, the signedness of the type might be important:

```
// smart_integer chooses appropriate signedness for results of arithmetic operations
auto a = smart_integer{7u};
auto b = smart_integer{-3};
auto c = a * b; // smart_integer<int>{-21}
```

To determine the signedness of `c`, `smart_integer` must know about the signedness of `a` and `b`. Because `a` and `b` are composed of fundamental types (unsigned `int` and signed `int`), `numeric_limits::is_signed` is already specialized for them. If they were custom numeric types, custom `numeric_limits` specializations would be required.

But `numeric_limits` contains only a few of the necessary attributes. For example, once `smart_integer` has determined that a signed type is required, it may need to convert an existing type:

```
auto m = smart_integer{5u};
auto s = smart_integer{10u};
auto d = m - s; // smart_integer<int>{-5}
```

Here, the representational type of `d` can be determined using `make_signed_t<uint32_t>` but `make_signed` must not be specialized for custom numeric types.

[P0437] proposes a new header, `<num_traits>`, containing free-standing equivalents of most of the attributes currently found in `<limits>`. Crucially, they are user-customizable, thus lifting a heavy restriction on most of the definitions in `<type_traits>`.

This proposal begins the task of extending the contents of `/` with definitions that make numeric types more generic. In doing so, it makes it possible for those types to be used to instantiate compositional numeric types.

A parallel can be found in existing support for iterators. Facilities such as `iterator_traits` and `advance` provide equivalence between custom integer types and raw pointers. This makes it possible to use efficient language-level features in expressive high-level abstractions.

Desirata

The following class templates are user-customizable. (Where appropriate, accompanying type aliases ending in `_t` and variable templates ending in `_v` are assumed.)

General Traits

The most common required traits are as follows.

num_digits - Determine the Number of Digits in a Given Type

```
template<class T>
struct num_digits;

static_assert(num_digits_v<int64_t> == 63);
```

This trait is a free equivalent of `numeric_limits<T>::digits` and is present in [P0437].

set_num_digits - Produce a Type With the Desired Number of Digits

```
template<class T, int MinDigits>
struct set_num_digits;

static_assert(std::is_same_v<set_num_digits_t<unsigned, 8>, std::uint8_t>);
```

This class complements `num_digits` and produces a type which is the same as the input except for its width. (Note: it replaces the `set_width` type proposed in [P0381].)

is_signed / make_signed / make_unsigned - Manipulate Signedness

These are all present in `<type_traits>` but user-specialization is not allowed. Either this restriction needs to be lifted or alternative definitions must be found.

Composition-specific Additions

Additional definitions which are only required for implementing compositional types are as follows.

is_composite - Determine if Type is Composite

```
template<class T>
struct is_composite;

static_assert(!is_composite_v<short>);
static_assert(is_composite_v<fixed_point<short>>);
```

This trait is necessary to distinguish between fundamental types which have no underlying representational value and custom types which conform to the proposed compositional approach.

to_rep - Extract Underlying Value

```
template<class T>
struct to_rep;

// a fundamental type is its own representation; it wraps nothing
long r = to_rep<long>()(1L);

// a compositional type is a value wrapper; to_rep unwraps it
long r = to_rep<smart_integer<long>>()(smart_integer<long>{1L});
```

This type is a function object - rather than a type trait. It is used to extract the underlying representational value from the composite type. For fundamental values, it simply returns that value.

If it were specialized for `chrono::duration`, it would take an object of type `chrono::duration` and return the result of `chrono::duration::count()`.

from_rep - Create from Underlying Value

```
template<class T>
struct from_rep;

// like to_rep, from_rep leaves its argument unchanged when instantiated with fundamental types
int i = from_rep<int>()(7);

// for compositional types, it can be used to create objects
auto f = from_rep<fixed_point<int, -1>>()(99);
// f is now fixed_point<int, -1>{49.5}
```

The complement to `to_rep`, this function object is a factory function.

from_value - Determine Type from Initial Value

```
template<class T, class Value>
struct from_value;

auto s = from_value_t<fixed_point<int16_t, -1>, unsigned long>{99UL};
// s is now fixed_point<unsigned long, 0>{99}
```

This type trait transforms `T` into whatever equivalent type best suits assignment from `Value`. In the case of composite types of `T`, that will typically mean replacing `T`'s representational type with `Value`.

In the above example, the fixed-point exponent is also set to zero. This reflects the fact that integers implicitly all have a radix position of zero.

Discussion

Relation to fixed_point

Many of the traits presented here were identified during the design of the `fixed_point` type proposed in [\[P0037\]](#). Without them, `fixed_point` can only be specialized for fundamental numeric types which greatly reduces its usefulness. However, this proposal is not a strict prerequisite for [\[P0037\]](#).

Missing Traits

Some numeric types which have been prototyped do not require all of the above traits. And it is likely that some numeric types - which have yet to be written - require traits which are not listed here. But it is hoped that this list is a good starting point and will allow users to create many of the types they desire.

Bikeshedding

All names in this document are placeholders. The choice of `set_digits` in particular is problematic but no satisfactory alternative has yet found consensus. Use of the term `rep` comes from `chrono::duration`.